

Relocalization

PHỤ THUỘC A/01-hình-học-da-thi (PnP — bước cuối của mọi pipeline); A/06-bat-dinh-va-lan-truyen-sai-so (hiệp phương sai để đặt ngưỡng chấp nhận); B/07-dac-trung-va-data-association (khớp điểm giữa hai khung rất xa nhau); B/11-loop-closure-va-nhat-quan-toan-cuc (cùng công nghệ phát hiện, cùng cấu trúc chỉ phí bất đối xứng).

TÓM TẮT

Có ba khoảnh khắc đặc biệt trong đời một hệ SLAM: lúc bắt đầu (B/10), lúc quay lại chỗ cũ (B/11), và lúc lạc đường — chương này. Relocalization trả lời câu hỏi “tôi đang ở đâu trong bản đồ này” khi không có tiên nghiệm nào, và nó là chương gần sản phẩm nhất trong Phần B vì nó quyết định điều mà khách hàng thật sự quan tâm: hệ thống có tự phục hồi được không, và mất bao lâu. Nội dung có bốn phần. Một: hai bài toán khác nhau thường bị gộp làm một — phục hồi sau mất bám ngắn, và định vị từ đầu trong một bản đồ đã có — với các yêu cầu rất khác nhau. Hai: kiến trúc phân cấp *truy hồi* → *khớp* → *PnP* đã trở thành chuẩn công nghiệp, và vì sao nó thắng. Ba: các hướng học được, gồm cả một trường hợp đáng học là hồi quy tư thế tuyệt đối — một ý tưởng hấp dẫn hoá ra không làm điều người ta tưởng. Bốn: kiểm định chấp nhận, với cùng lập luận bất đối xứng như B/11 nhưng gay gắt hơn. Nếu chỉ nhớ một điều: *một relocalization sai hại hơn không relocalization* — vì hệ thống đang lạc thì biết mình lạc, còn hệ thống tin sai thì không.

KIẾN THỨC CẦN CÓ TRƯỚC

- **relocalization** — xác định tư thế camera trong một bản đồ đã có, không dùng tiên nghiệm về vị trí. Khác với tracking ở chỗ tracking luôn có dự đoán từ khung trước.
- **kidnapped robot** — bài toán kinh điển: robot bị nhắc lên và đặt ở một chỗ khác trong khi tắt nguồn, rồi bật lên và phải tự tìm ra mình ở đâu. Tên gọi cho trường hợp khó nhất của relocalization.
- **cold start** — khởi động hệ thống trong một bản đồ đã lưu, không biết vị trí ban đầu. Cùng bài toán với kidnapped robot nhưng thường có ràng buộc thời gian gắt hơn vì người dùng đang chờ.
- **localization phân cấp (hierarchical localization)** — kiến trúc ba bước: truy hồi vài ứng viên toàn cục, khớp đặc trưng cục bộ với chúng, rồi PnP. Tách bài toán lớn thành hai bài toán nhỏ hơn mà mỗi cái đều có công cụ tốt.
- **scene coordinate regression** — học một hàm ánh xạ trực tiếp từ một mảng ảnh sang tọa độ 3D của nó trong bản đồ, rồi dùng PnP trên các tương ứng thu được. Bỏ qua bước truy hồi và bước mô tả.
- **hồi quy tư thế tuyệt đối (absolute pose regression, APR)** — học một mạng ánh xạ thẳng từ ảnh sang tư thế 6-DoF. Ý tưởng đơn giản nhất có thể, và mục 4B giải thích vì sao nó không hoạt động như kỳ vọng.

CÂU HỎI MỞ ĐẦU

- Q1.** Hệ thống mất bám. Nó có thể (a) tiếp tục báo tư thế cuối cùng biết được, (b) báo “tôi không biết”, hoặc (c) relocalize và báo một tư thế mới. Xếp ba lựa chọn theo mức nguy hiểm, và giải thích thứ tự.

- Q2. HLoc tách bài toán thành truy hồi rồi khớp rồi PnP, thay vì khớp trực tiếp với toàn bộ bản đồ. Nêu hai lý do khác nhau vì sao việc tách đó thắng — một lý do về chi phí và một lý do về chất lượng.
- Q3. PoseNet học ánh xạ thẳng từ ảnh sang tư thế. Nó hoạt động trên tập kiểm tra. Nêu điều nó thực sự học được, và vì sao điều đó khác với điều người ta tưởng nó học được.
- Q4. Robot lau nhà bật lên trong một căn hộ đã được lập bản đồ tuần trước. Từ tuần trước, một cái ghế đã bị dịch và rèm cửa đã được kéo. Nêu ba thứ có thể làm relocalization thất bại, xếp theo mức khó xử lý.
- Q5. Relocalization của bạn trả về một tư thế với hai trăm inlier PnP. Nêu ba phép kiểm bổ sung trước khi chấp nhận nó, và nói mỗi phép bắt được lỗi gì.

1.1 Đặt vấn đề

Mọi hệ SLAM mất bám. Không phải nếu mà là khi nào: camera bị che, ánh sáng tắt, robot bị đẩy, chuyển động quá nhanh, một người đi ngang chắn hết khung hình. Câu hỏi thiết kế không phải làm sao để không bao giờ mất bám — điều đó bất khả — mà là *chuyện gì xảy ra sau đó*.

Bài toán gốc: cho một ảnh và một bản đồ, xác định tư thế camera mà không có tiên nghiệm nào về vị trí. Nghe giống loop closure, và công nghệ thì giống thật, nhưng bối cảnh khác ở một điểm quyết định: trong loop closure, hệ thống có một ước lượng vị trí (dù trôi) để kiểm chéo; trong relocalization, nó không có gì.

Mất đi phép kiểm chéo ấy là mất tầng phòng thủ thứ ba ở B/11 mục 3B — phép kiểm “có nhất quán với chỗ tôi nghĩ mình đang đứng không”. Nên relocalization khó hơn loop closure, dù dùng cùng công cụ.

Ai gặp bài toán này? Ba nhóm với ba yêu cầu khác nhau, và gộp chúng làm một là một lỗi thiết kế phổ biến.

Nhóm một — phục hồi sau mất bám ngắn. Hệ đang chạy, mất bám vài giây. Nó có một ước lượng thô từ trước khi mất, và nó chỉ cần tìm lại trong một vùng nhỏ. Yêu cầu: rất nhanh (người dùng không nên nhận ra), và có thể dùng tiên nghiệm.

Nhóm hai — cold start trong bản đồ đã có. Robot bật lên, không biết mình ở đâu. Không có tiên nghiệm, phải tìm trong toàn bộ bản đồ. Yêu cầu: nhanh vừa phải (vài giây chấp nhận được), và *đúng* — một khởi động sai vị trí là một lỗi mà khách hàng nhìn thấy ngay.

Nhóm ba — localization thuần túy. Không có SLAM, chỉ có một bản đồ lớn đã dựng sẵn và một ảnh cần định vị. Đây là bài toán của thị giác máy tính (định vị ảnh trong thành phố) hơn là của robot, và nó có quy mô lớn hơn nhiều — hàng triệu ảnh trong bản đồ.

Ba nhóm chia nhau công nghệ nhưng khác nhau về ràng buộc, và một hệ sản phẩm thường cần cả nhóm một lẫn nhóm hai với hai đường xử lý khác nhau.

Trước khi có kiến trúc phân cấp, cách làm là khớp đặc trưng của ảnh truy vấn với *toàn bộ* bản đồ. Với bản đồ vài nghìn landmark thì được; với bản đồ vài triệu thì không — cả về chi phí lẫn về chất lượng, vì tỉ lệ khớp sai tăng theo kích thước không gian tìm kiếm. Đây là câu trả lời cho câu hỏi mở đầu thứ hai và là lý do kiến trúc ba bước tồn tại.

TRỰC GIÁC

Bạn tỉnh dậy trong một thành phố bạn đã đi qua, không nhớ mình đang ở đâu, và có một cuốn sổ đầy ảnh chụp các góc phố bạn từng đi.

Cách ngây thơ: giờ từng trang, so ảnh trong sổ với cảnh trước mặt. Với sổ mười trang thì được. Với sổ mười nghìn trang thì hết ngày — và tệ hơn, xác suất bạn gặp một trang *trông giống* mà không phải đúng chỗ tăng theo số trang.

Cách người ta thật sự làm có hai bước. Đầu tiên nhìn tổng thể: kiến trúc kiểu gì, cây gì, màu tường ra sao — và lật nhanh tới vài trang có *ấn tượng chung* giống nhất. Rồi với vài trang đó, so kỹ từng chi tiết: cái biển hiệu này, vết nứt kia.

Hai bước ấy dùng hai loại thông tin khác nhau. Bước một dùng *ấn tượng toàn cục* — thô, nhưng so được rất nhanh với mười nghìn trang. Bước hai dùng *chi tiết cục bộ* — chính xác, nhưng chỉ so được với vài trang.

Đó là kiến trúc phân cấp, và nó thắng vì hai lý do chứ không phải một. Rõ ràng là nó *nhANH hơn*. Ít rõ hơn nhưng quan trọng không kém: nó *chính xác hơn*, vì khi chỉ so chi tiết với ba trang thay vì mười nghìn, cơ hội nhầm giảm hàng nghìn lần.

Còn một chuyện nữa, và nó là chuyện quan trọng nhất của chương. Giả sử sau cả hai bước bạn vẫn không chắc — có hai góc phố trông giống nhau. Bạn có hai lựa chọn: đoán một cái và đi tiếp, hoặc thừa nhận “tôi chưa biết” và đứng nhìn thêm.

Đoán và đi tiếp nguy hiểm hơn nhiều. Vì một khi đã đi, bạn sẽ diễn giải mọi thứ thấy sau đó theo giả định sai, và bạn càng lúc càng tin vào nó. Còn nếu bạn thừa nhận chưa biết, ít nhất bạn vẫn đang tìm.

1.2 Kiến trúc phân cấp

A. Ba bước

HLoc [1] hệ thống hoá kiến trúc đã trở thành chuẩn:

Bước một — truy hồi toàn cục. Tính một biểu diễn toàn cục cho ảnh truy vấn (BoW, NetVLAD, hoặc tương đương) và lấy k khung giống nhất trong bản đồ, với k thường vài chục. Chi phí gần hằng số nhờ chỉ mục.

Bước hai — khớp cục bộ. Với mỗi ứng viên, khớp đặc trưng cục bộ giữa ảnh truy vấn và khung ấy (B/07). Vì chỉ có vài chục ứng viên, ta có thể dùng matcher đắt như SuperGlue mà vẫn chấp nhận được.

Bước ba — PnP. Các khớp cho tương ứng 2D–3D với landmark của bản đồ; chạy PnP trong RANSAC (A/01 mục 5) để lấy tư thế và tập inlier.

B. Vì sao tách lại thẳng

Hai lý do độc lập, và lý do thứ hai quan trọng hơn nhưng ít được nhắc.

Chi phí: bước một là $O(1)$ nhờ chỉ mục ngược; bước hai là $O(k)$ với k nhỏ và không phụ thuộc kích thước bản đồ. Tổng chi phí gần như không tăng khi bản đồ lớn lên — một tính chất thiết yếu cho hệ chạy lâu.

Chất lượng: đây là lý do tinh tế hơn. Khi khớp một đặc trưng với toàn bộ bản đồ, số ứng viên sai tăng tuyến tính theo kích thước bản đồ, nên xác suất một khớp sai lọt qua ratio test cũng tăng. Với mười nghìn landmark, ratio test đủ tốt; với mười triệu, nó không còn đủ. Bằng cách giới hạn tìm kiếm vào vài chục khung, ta giữ không gian tìm kiếm nhỏ và ratio test lấy lại hiệu lực. Nói cách khác: *bước truy hồi không chỉ tăng tốc mà còn khôi phục điều kiện hoạt động của bước khớp.*

Điều này là một trường hợp của một nguyên tắc lặp lại trong cả cây: *thu hẹp không gian tìm kiếm là cách rẻ nhất để cải thiện data association* — cùng nguyên tắc với tiên nghiệm chuyển động ở B/07 mục 5C.

C. Dùng tiên nghiệm thô khi có

Với nhóm một — phục hồi sau mất bám nắm — ta có tiên nghiệm, và bỏ qua nó là lãng phí.

Tư thế cuối cùng biết được, cộng với thời gian đã trôi qua và một giới hạn vận tốc, cho một vùng khả dĩ. Chỉ tìm trong vùng đó: rẻ hơn nhiều, và tỉ lệ dương tính giả giảm mạnh vì phần lớn các ứng viên gây nhầm lẫn nằm ngoài vùng.

Các nguồn tiên nghiệm thô khác đáng biết trong sản phẩm: odometry bánh xe tiếp tục chạy trong lúc thị giác mất bám; IMU cho hướng và một ước lượng dịch chuyển ngắn hạn; wifi hoặc bluetooth beacon cho vị trí ở mức phòng; một marker trong tầm nhìn cho vị trí tuyệt đối (`.../marker/`). Mỗi nguồn thu hẹp không gian tìm kiếm, và chúng *cộng dồn*.

Đây là lý do thực dụng quan trọng nhất để có nhiều cảm biến, và nó khác với lý do thường được nêu (độ chính xác cao hơn). Cảm biến phụ có giá trị lớn nhất không phải khi mọi thứ chạy tốt mà khi thị giác thất bại — chúng cho hệ thống một chỗ đứng để phục hồi.

1.3 Các hướng học được

A. Scene coordinate regression

Thay vì truy hồi rồi khớp, học một hàm ánh xạ trực tiếp từ một mảng ảnh sang tọa độ 3D của nó trong bản đồ. DSAC* [2] và ACE [3] theo hướng này: mạng cho mỗi mảng một tọa độ 3D, và PnP trong RANSAC chạy trên các tương ứng đó.

Ưu điểm: bỏ được cả bước truy hồi lẫn bước mô tả; và vì mạng đã học cảnh cụ thể, nó xử lý được các vùng mà descriptor tổng quát thất bại. ACE còn giảm thời gian huấn luyện cho một cảnh mới xuống mức phút, thay vì giờ.

Giới hạn quan trọng: mô hình *gắn với một cảnh cụ thể*, phải huấn luyện lại cho mỗi bản đồ. Với ứng dụng có một bản đồ cố định — một nhà máy, một bảo tàng — điều đó chấp nhận được. Với robot phải hoạt động ở nhà bất kỳ khách hàng nào, nó là một rào cản triển khai thật: mỗi lần lập bản đồ lại là một lần huấn luyện lại, và điều đó phải chạy trên thiết bị hoặc trên đám mây với mọi hệ quả về quyền riêng tư (C/13).

B. Hồi quy tư thế tuyệt đối, và vì sao nó không làm điều ta tưởng

Đây là câu trả lời cho câu hỏi mở đầu thứ ba, và tôi đưa nó vào vì nó là một bài học phương pháp có giá trị vượt ra ngoài chủ đề.

Ý tưởng đơn giản nhất có thể: học một mạng ánh xạ thẳng từ ảnh sang sáu (hoặc bảy) con số tư thế. PoseNet [4] làm điều đó và hoạt động — trên tập kiểm tra.

Sattler và cộng sự [5] đặt câu hỏi mà lẽ ra phải đặt sớm hơn: *mạng thực sự học được gì?* Kết quả của họ là mạng hành xử gần với *nội suy giữa các tư thế huấn luyện* hơn là với một bộ ước lượng hình học. Cụ thể, độ chính xác của nó không vượt hơn đáng kể so với một đường cơ sở rất đơn giản: truy hồi ảnh huấn luyện giống nhất rồi trả về tư thế của ảnh đó.

Điều đó có nghĩa gì? Mạng học được một ánh xạ *từ vẻ ngoài sang tư thế* trên tập huấn luyện, và nó nội suy giữa chúng. Nó *không* học hình học chiều, nên nó không tổng quát hoá ra các tư thế xa tập huấn luyện — đúng chỗ mà một bộ ước lượng hình học lại mạnh nhất.

Bài học phương pháp đáng rút ra, và nó áp dụng rộng hơn nhiều so với chủ đề này: *một phương pháp đạt điểm số tốt trên benchmark không chứng minh rằng nó làm điều ta nghĩ nó làm*. Cách kiểm là so với một đường cơ sở tầm thường thiết kế để có cùng khả năng nội suy nhưng không có gì khác. Nếu phương pháp mới không thắng đường cơ sở ấy, thì phần “mới” chưa được chứng minh. Đây là đúng tinh thần của `research-rules.md` mục 7 về việc phải làm baseline từ tế trước, và đó là lý do tôi đưa trường hợp này vào một chương học tập.

CẠM BẤY

Tôi không dẫn số liệu benchmark relocalization ở đây, và cần nói rõ lý do.

Các con số trên Aachen Day-Night, 7-Scenes hay Cambridge phụ thuộc cực mạnh vào: bản đồ được dựng thế nào (SfM từ chính tập huấn luyện hay từ nguồn khác), ngưỡng nào được coi là “thành công” (5 cm/5° hay 25 cm/2° — các bài khác nhau dùng khác nhau), và có bao nhiêu ảnh trong bản đồ. Hai bài báo báo cáo cho cùng phương pháp trên cùng bộ dữ liệu thường không so sánh được.

Nghiêm trọng hơn, với ứng dụng robot thì các benchmark ấy *đo sai thứ*. Chúng đo độ chính xác trung bình trên các truy vấn có câu trả lời trong bản đồ. Chúng không đo: tỉ lệ trả về một tư thế *sai* một cách tự tin; hành vi khi truy vấn *không* có trong bản đồ; và thời gian xấu nhất. Với sản phẩm, ba thứ đó quyết định hơn.

Cách đúng, và nó lặp lại lời khuyên ở B/07: đo trên dữ liệu của chính bạn, trong chính điều kiện triển khai, và đo *tỉ lệ dương tính giả ở ngưỡng bạn định dùng* chứ không phải độ chính xác trung bình. C/18 bàn cách thiết kế phép đo đó.

1.4 Kiểm định chấp nhận

A. Vì sao reloc sai hại hơn không reloc

Đây là câu trả lời cho câu hỏi mở đầu thứ nhất, và là luận điểm trung tâm của chương.

Xếp ba lựa chọn theo mức nguy hiểm, từ ít tới nhiều:

(b) Báo “*tôi không biết*” — an toàn nhất. Hệ thống ở trạng thái đã biết là xấu, và mọi module phía trên có thể phản ứng phù hợp: robot dừng lại, người dùng được thông báo, một chiến lược phục hồi được kích hoạt.

(a) *Tiếp tục báo tư thế cuối cùng* — nguy hiểm vừa. Tư thế ngày càng sai theo thời gian, nhưng sai một cách *dự đoán được*: nó cũ, và nếu hệ thống ghi lại độ tuổi của ước lượng thì module phía trên biết mà chiết khấu.

(c) *Relocalize sai* — nguy hiểm nhất. Hệ thống báo một tư thế *mới*, với độ tin cậy cao, và hoàn toàn sai. Mọi thứ phía trên tin nó. Tệ hơn cả: hệ thống sẽ bắt đầu tracking từ tư thế sai đó, khớp landmark của vùng sai, và *củng cố* niềm tin sai — vì các quan sát mới sẽ khớp với giả thuyết sai một cách nhất quán trong một thời gian. Nếu nó còn ghi cập nhật vào bản đồ thì bản đồ cũng bị hỏng.

Chênh lệch giữa (b) và (c) là lý do chương này tồn tại. Và nó gay gắt hơn B/11 vì ở đó, hệ thống ít nhất còn một ước lượng để kiểm chéo; ở đây nó không có gì.

B. Bốn phép kiểm

Đây là câu trả lời cho câu hỏi mở đầu thứ năm.

Một — *số và tỉ lệ inlier PnP*. Cần thiết, không đủ; cùng lý do ở A/01 mục 6. Đặt ngưỡng cao hơn nhiều so với tracking.

Hai — phân bố không gian của inlier. Hai trăm inlier tụ ở một góc ảnh cho một tư thế có bất định lớn theo vài hướng, dù số đếm trông tốt. Kiểm bằng diện tích bao lồi của inlier hoặc bằng số ô lưới có inlier — rẻ và hiệu quả, và nó là B/07 mục 4A áp dụng ở đây.

Ba — hiệp phương sai của tư thế. Tính Λ của bài toán PnP và xem phổ của nó (A/06). Đây là phép kiểm đúng về nguyên tắc vì nó đo trực tiếp mức xác định. Giới hạn: nó chịu bệnh quá tự tin, nên ngưỡng phải bảo thủ.

Bốn — nhất quán thời gian. Đây là phép kiểm mạnh nhất và rẻ bất ngờ: *đừng chấp nhận một relocalization từ một khung duy nhất*. Yêu cầu vài khung liên tiếp cùng cho tư thế nhất quán với nhau và nhất quán với chuyển động đo được giữa chúng (từ odometry, IMU, hay optical flow). Một dương tính giả hiếm khi lặp lại nhất quán qua nhiều khung với chuyển động đúng.

Cái giá của phép kiểm bốn là độ trễ — vài trăm mili giây. Với nhóm hai (cold start) đó là chấp nhận được và nên làm mặc định. Với nhóm một (phục hồi nhanh) thì cần cân nhắc, và đó là một lý do nữa để tách hai đường xử lý.

C. Ba chế độ hồng trong sản phẩm

Đây là câu trả lời cho câu hỏi mở đầu thứ tư, xếp theo mức khó xử lý.

Dễ nhất — thay đổi ánh sáng. Rèm kéo, đèn bật. Descriptor cổ điển kém bền với điều này; place recognition học được bền hơn nhiều. Có lời giải, dù tốn tài nguyên.

Khó vừa — vật thể dịch chuyển. Cái ghế đã dịch. Các landmark trên nó giờ ở sai chỗ, và chúng sẽ là outlier trong PnP. RANSAC xử lý được nếu chúng là thiểu số. Vấn đề thật là *tích lũy*: sau vài tháng, một tỉ lệ đáng kể bản đồ đã lỗi thời, và có lúc thiểu số thành đa số. Cần cơ chế cập nhật bản đồ (C/13), và đó là bài toán chưa được giải tốt.

Khó nhất — thay đổi cấu trúc. Bàn ghế được sắp lại hoàn toàn, tường được sơn lại, một phòng được cải tạo. Bản đồ cũ không còn mô tả thế giới. Không phép kiểm nào phân biệt được “tôi đang ở chỗ khác” với “tôi ở đúng chỗ nhưng chỗ này đã đổi”, và hai kết luận đòi hai hành động ngược nhau. Đây là bài toán *lifelong mapping*, và D/22 xếp nó là một trong tám khoảng trống lớn nhất.

CẦN NHỚ

Một hệ relocalization dùng được trong sản phẩm phải trả lời được ba câu, không phải một.

Câu một: *tôi ở đâu?* — đây là câu mà mọi bài báo trả lời.

Câu hai: *tôi có chắc không?* — một tư thế kèm hiệp phương sai đáng tin, hoặc ít nhất một mức tin cậy đã được hiệu chuẩn trên dữ liệu thật. Rất ít công trình trả lời câu này.

Câu ba: *tôi có nên nói gì không?* — tức khả năng trả về “không biết”. Đây là câu quan trọng nhất cho sản phẩm và là câu ít được nghiên cứu nhất.

Ba câu ấy tương ứng với ba chỉ tiêu đo khác nhau, và một hệ tối ưu cho câu một có thể rất tệ ở câu ba. Khi đánh giá, hãy đo cả ba — và với sản phẩm, hãy đặt chỉ tiêu cho câu ba trước, rồi tối ưu câu một trong ràng buộc đó.

Kiến thức này đang sống ở đâu (công trình đang dùng chính ý tưởng của chương; chốt 9/2026)

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ORB-SLAM3 [6]	Reloc bằng DBoW2 + PnP RANSAC + tối ưu tư thế; yêu cầu nhất quán qua nhiều khung	Cài đặt của mục 2 và phép kiểm bốn ở mục 4B
HLoc [1]	NetVLAD/SuperPoint/SuperGlue + PnP; pipeline chuẩn công nghiệp	Kiến trúc phân cấp đầy đủ; đọc để thấy mỗi bước tách bạch thể nào

Công trình / hệ thống	Dùng ý tưởng nào của chương	Ghi chú
ACE [3]	Scene coordinate regression với huấn luyện nhanh cho cảnh mới	Mục 3A; chú ý rằng buộc “một mô hình một cảnh”
[5]	Phân tích cho thấy APR gần với truy hồi ảnh hơn là với ước lượng hình học	Mục 3B — bài học phương pháp quan trọng hơn kết quả cụ thể
RTAB-Map	Reloc và quản lý bộ nhớ cho bản đồ dài hạn	Một trong ít hệ mã nguồn mở nghĩ về hoạt động nhiều phiên

MINI PROJECT SLAM · Đo tỉ lệ dương tính giả của relocalization, rồi hạ nó xuống bằng bốn phép kiểm · 3 ngày

Mục tiêu	chuyển từ “hệ của tôi relocalize được” sang “hệ của tôi trả về tư thế sai $X\%$ số lần ở ngưỡng Y , và nói không biết $Z\%$ ”.
Dữ liệu	một chuỗi có chân trị để dựng bản đồ, và một chuỗi thứ hai cùng môi trường để truy vấn. Quan trọng: thêm <i>các truy vấn không có câu trả lời</i> — ảnh từ một phòng khác không có trong bản đồ. Phần này là phần bị bỏ qua trong mọi benchmark và là phần quyết định trong sản phẩm.
Các bước	cài pipeline ba bước ở mục 2A; chạy trên cả truy vấn có và không có câu trả lời; với ngưỡng inlier quét qua một dải, vẽ ba đường cong theo ngưỡng — tỉ lệ đúng, tỉ lệ sai, tỉ lệ “không biết”; rồi thêm từng phép kiểm ở mục 4B và vẽ lại; cuối cùng đo độ trễ thêm vào bởi phép kiểm bốn.
Xong khi	bạn có một bảng: mỗi phép kiểm giảm tỉ lệ dương tính giả bao nhiêu, tăng tỉ lệ “không biết” bao nhiêu, và tốn thêm bao nhiêu mili giây. Bạn chọn được một điểm làm việc <i>từ yêu cầu sản phẩm</i> — “tôi chấp nhận sai không quá $0,1\%$ ” — chứ không từ một ngưỡng chép ở đâu. Và bạn có con số cho các truy vấn không có câu trả lời, thứ mà không benchmark nào cho bạn.
Nối với	C/13-quan-ly-ban-do xử lý vấn đề bản đồ lỗi thời ở mục 4C; D/21-tu-prototype-len-mass-production biến điểm làm việc này thành một chỉ tiêu nghiệm thu.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Từ an toàn tới nguy hiểm: (b) báo “không biết” — hệ ở trạng thái đã biết là xấu, mọi module phía trên phản ứng được; (a) báo tư thế cũ — sai dần nhưng sai *dự đoán được*, và nếu ghi kèm độ tuổi thì phía trên chiết khấu được; (c) reloc sai — nguy hiểm nhất vì hệ báo một tư thế *mới* với độ tin cậy cao và hoàn toàn sai. Tệ hơn cả, nó sẽ bắt đầu tracking từ đó, khớp landmark vùng sai, và *củng cố* niềm tin sai vì các quan sát mới khớp nhất quán với giả thuyết sai trong một thời gian. Nếu còn ghi vào bản đồ thì bản đồ cũng hỏng.
- A2. Lý do *chi phí*: bước truy hồi là $O(1)$ nhờ chỉ mục, bước khớp là $O(k)$ với k nhỏ và không phụ thuộc kích thước bản đồ — nên tổng chi phí gần không đổi khi bản đồ lớn lên. Lý do *chất lượng*, tinh tế hơn và quan trọng hơn: khi khớp với toàn bộ bản đồ, số ứng viên sai tăng tuyến tính theo kích thước, nên xác suất một khớp sai lọt qua ratio test cũng tăng — với mười triệu landmark, ratio test không còn đủ. Giới hạn tìm kiếm vào vài chục khung giữ không gian nhỏ và khôi phục điều kiện hoạt động của bước khớp. Đây là cùng nguyên tắc với tiên nghiệm chuyển động ở B/07 mục 5C.
- A3. Nó học một ánh xạ *từ vẻ ngoài sang tư thế* trên tập huấn luyện, rồi *nội suy* giữa chúng — chứ không học hình học chiều. Sattler và cộng sự [5] cho thấy nó không vượt hơn đáng kể một đường cơ sở rất đơn giản là truy hồi ảnh huấn luyện giống nhất rồi trả về tư thế của ảnh đó. Hệ quả: nó không tổng quát hoá ra các tư thế xa tập huấn luyện — đúng chỗ mà bộ ước lượng hình học mạnh nhất. Bài học phương pháp rộng hơn: điểm số tốt trên benchmark không chứng minh phương pháp làm điều ta nghĩ nó làm; cách kiểm là so với một đường cơ sở tầm thường có cùng khả năng nội suy nhưng không có gì khác.
- A4. Từ dễ tới khó: (a) *thay đổi ánh sáng* (rèm kéo) — descriptor cổ điển kém bền, place recognition học được bền hơn nhiều; có lời giải. (b) *vật thể dịch chuyển* (cái ghế) — landmark trên nó thành outlier, RANSAC xử lý được khi chúng là thiểu số; vấn đề thật là *tích lũy* qua nhiều tháng cho

tới khi thiếu số thành đa số, và cần cơ chế cập nhật bản đồ. (c) *thay đổi cấu trúc* — không phép kiểm nào phân biệt được “tôi ở chỗ khác” với “tôi ở đúng chỗ nhưng chỗ này đã đổi”, và hai kết luận đòi hai hành động ngược nhau. Đây là bài toán lifelong mapping, một trong tám khoảng trống ở D/22.

- A5. Bốn phép: (1) *số và tỉ lệ inlier* — cần nhưng không đủ, ngưỡng cao hơn tracking nhiều. (2) *phân bố không gian của inlier* — hai trăm inlier tụ một góc cho tư thế bất định theo vài hướng dù số đếm tốt; kiểm bằng số ô lưới có inlier. (3) *hiệp phương sai từ Λ của bài toán PnP* — đúng về nguyên tắc vì nó đo trực tiếp mức xác định, nhưng chịu bệnh quá tự tin nên ngưỡng phải bảo thủ. (4) *nhất quán thời gian* — mạnh nhất và rẻ bất ngờ: đừng chấp nhận từ một khung duy nhất, yêu cầu vài khung liên tiếp cho tư thế nhất quán với nhau và với chuyển động đo được giữa chúng. Cái giá của (4) là vài trăm mili giây độ trễ.

NÓI LẠI THẬT ĐƠN GIẢN

Bạn tỉnh dậy giữa một thành phố từng đi qua, không nhớ mình ở đâu, và có một cuốn sổ ảnh các góc phố từng đi.

Cách làm hợp lý có hai bước. Đầu tiên nhìn tổng thể — kiến trúc, cây cối, màu tường — rồi lật nhanh tới vài trang có ấn tượng chung giống nhất. Rồi với vài trang đó, so kỹ từng chi tiết.

Hai bước, hai loại thông tin. Ấn tượng chung thì thô nhưng so được với mười nghìn trang trong vài giây. Chi tiết thì chính xác nhưng chỉ so được với vài trang. Ghép hai lại thì vừa nhanh vừa ít nhầm — và cái “ít nhầm” quan trọng hơn cái “nhanh”, vì so chi tiết với ba trang thay vì mười nghìn thì cơ hội nhầm giảm hàng nghìn lần.

Nhưng chuyện quan trọng nhất của chương không phải cách tìm. Đó là chuyện gì xảy ra khi bạn *không chắc*.

Giả sử có hai góc phố trông giống nhau và bạn không phân biệt được. Hai lựa chọn: đoán một cái rồi đi tiếp, hoặc nói “tôi chưa biết” rồi đứng nhìn thêm một lúc.

Đoán rồi đi tiếp tệ hơn nhiều, và không phải vì lý do hiển nhiên. Một khi đã tin mình ở góc phố A, bạn sẽ diễn giải *mọi thứ* thấy sau đó theo giả định ấy. Cái cửa hàng kia hơi khác? Chắc họ đổi biển. Con đường không rõ như mong đợi? Chắc mình nhớ nhầm. Bạn càng đi càng tin, và càng khó quay lại. Trong khi nếu bạn thừa nhận chưa biết, ít nhất bạn vẫn đang tìm.

Cái máy cũng hệt vậy, và tệ hơn: nó không có cảm giác gợn khi mọi thứ hơi không khớp. Nó chỉ thấy con số phù hợp đủ tốt và đi tiếp.

Nên câu hỏi thiết kế đúng không phải “làm sao để máy luôn tìm được chỗ đúng” mà là “làm sao để máy biết nói tôi chưa biết”. Và cách làm, hoá ra, đơn giản đến bất ngờ: *đừng quyết định từ một cái nhìn duy nhất*. Nhìn thêm vài giây, đi thêm vài bước, và kiểm xem những gì thấy được có khớp với cả việc mình vừa di chuyển thế nào không. Một sự nhầm lẫn hiếm khi sống sót qua phép kiểm đó.

Chỗ không rút gọn được: chỗ không rút gọn được là khi *chỗ cũ đã thay đổi*. Bạn về đúng nhà mình nhưng ai đó đã kê lại toàn bộ đồ đạc. Máy không có cách nào phân biệt “tôi đang ở nhầm nhà” với “tôi ở đúng nhà nhưng nhà đã đổi” — và hai kết luận đòi hai hành động trái ngược. Đó là bài toán khó nhất trong triển khai dài hạn, và nó chưa có lời giải tốt.

Một câu: Một hệ đang lạc thì biết mình lạc; một hệ tin sai thì không — và đó là toàn bộ lý do phải biết nói “chưa biết”.

I 1.5 Điều gì chứng minh cách hiểu này sai

Mệnh đề chính — *reloc sai hại hơn không reloc* — là một lập luận cấu trúc dựa trên việc hệ thống sẽ củng cố niềm tin sai. Nó bị bác bỏ nếu một hệ có cơ chế *phát hiện và huỷ* một relocalization sai sau khi đã chấp nhận, đủ tin cậy để chi phí của (c) không còn vượt (b). Cơ chế như vậy có thể tồn tại — theo dõi tỉ lệ tracking và phần dư trong vài giây sau reloc, và huỷ nếu chúng xấu đi — và nó nên được cài. Tôi ngờ rằng nó không bắt được trường hợp khó nhất, là reloc vào một vùng *thật sự giống*, vì ở đó tracking sẽ chạy tốt một thời gian. Nhưng tôi chưa đo (*tôi suy ra, chưa đổi chiều nguồn*).

Mệnh đề thứ hai — *kiến trúc phân cấp thắng vì cả chi phí lẫn chất lượng* — kiểm được bằng cách so pipeline ba bước với khớp trực tiếp toàn bản đồ, ở nhiều kích thước bản đồ. Nó bị bác bỏ nếu chất lượng không khác nhau ở bản đồ nhỏ — điều tôi *dự đoán* là đúng, vì lập luận về chất lượng chỉ có hiệu lực khi bản đồ đủ lớn. Kiểm điều đó sẽ làm rõ ngưỡng kích thước mà kiến trúc phân cấp bắt đầu đáng giá.

Mệnh đề thứ ba — *APR hành xử như nội suy chứ không như ước lượng hình học* — là kết quả chính của [5] và là một mệnh đề đã được kiểm bởi người khác (cửa c). Nó có thể đã thay đổi với các kiến trúc mới hơn, và tôi chưa khảo sát tài liệu sau 2019 về điểm này. Đây là một mục trong 99-chua-biet.md.

Một mệnh đề mềm: chương này khẳng định *benchmark relocalization* đo sai thứ cho ứng dụng robot. Đó là một phán đoán về mức phù hợp của phép đo, không phải một sự kiện. Nó bị bác bỏ nếu ai đó cho thấy thứ hạng trên benchmark tương quan chặt với tỉ lệ dương tính giả trong triển khai thật. Tôi cho là không, vì benchmark không chứa truy vấn ngoài bản đồ, nhưng tôi không có dữ liệu.

1.6 Kết nối

Nổi lên trên. B/11-loop-closure-va-nhat-quan-toan-cuc chia sẻ gần như toàn bộ công nghệ và toàn bộ cấu trúc lập luận; khác biệt duy nhất và quyết định là ở đây không có ước lượng hiện tại để kiểm chéo, nên mất một tầng phòng thủ. B/07-dac-trung-va-data-association cho bước khớp và cho nguyên tắc thu hẹp không gian tìm kiếm. A/01-hinh-hoc-da-thi cho PnP. A/06-bat-dinh-va-lan-truyen-sai-so cho phép kiểm thứ ba và cảnh báo về giới hạn của nó.

Nổi xuống dưới. C/13-quan-ly-ban-do là chương xử lý ba vấn đề mà chương này nêu ra nhưng không giải: bản đồ lỗi thời theo thời gian, kích thước bản đồ tăng vô hạn, và việc ghép nhiều phiên. C/18-danh-gia-va-phuong-phap-luan chuẩn hoá cách đo ba câu hỏi ở khối *Cần nhớ*, đặc biệt câu ba vốn không có benchmark nào đo.

Nổi xa hơn. D/19-ben-vung-trong-the-gioi-thuc bàn về ba chế độ hồng ở mục 4C một cách đầy đủ hơn. D/21-tu-prototype-len-mass-production là chỗ chương này quan trọng nhất: khả năng tự phục hồi quyết định MTBF cảm nhận được của sản phẩm, và thời gian cold start là thứ khách hàng đo bằng đồng hồ. D/22-huong-nghien-cuu-con-trong-lay-lifelong-mapping — bài toán ở mục 4C — làm một trong tám khoảng trống.

Nổi ngang: ../marker/ cung cấp giải pháp thực dụng nhất cho cold start trong môi trường có thể sửa đổi: một marker trong tầm nhìn cho tư thế tuyệt đối ngay lập tức, với độ tin cậy cao hơn nhiều so với bất kỳ phương pháp thuần thị giác nào. Cái giá là phải đặt và bảo trì marker.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Xếp ba lựa chọn khi mất bám — bám tư thế cũ, bám không biết, relocalize — theo mức nguy hiểm, và giải thích cơ chế làm lựa chọn nguy hiểm nhất trở nên nguy hiểm.
2. Nêu hai lý do độc lập vì sao kiến trúc phân cấp thắng, và nói lý do nào chỉ có hiệu lực khi bản đồ đủ lớn.
3. PoseNet thực sự học được gì? Nêu đường cơ sở tầm thường dùng để phát hiện điều đó, và bài học phương pháp rộng hơn.
4. Nêu bốn phép kiểm trước khi chấp nhận một relocalization, và nói phép nào mạnh nhất và vì sao nó rẻ bất ngờ.
5. Ba chế độ hồng khi bản đồ cũ đi. Vì sao chế độ thứ ba không có lời giải tốt, và hai kết luận nào bị lẫn lộn?

6. Ba câu hỏi mà một hệ relocalization sản phẩm phải trả lời. Câu nào ít được nghiên cứu nhất, và vì sao nó quan trọng nhất?
7. Vì sao benchmark relocalization đo sai thứ cho ứng dụng robot? Nêu ba thứ chúng không đo.

Nguồn

- [1] P.-E. Sarlin, C. Cadena, R. Siegwart, et al., “From Coarse to Fine: Robust Hierarchical Localization at Large Scale,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A, 2019.
- [2] E. Brachmann and C. Rother, “Visual Camera Re-Localization from RGB and RGB-D Images Using DSAC,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, no. 9, pp. 5847–5865, 2022, mức tin B.
- [3] E. Brachmann, T. Cavallari, and V. A. Prisacariu, “Accelerated Coordinate Encoding: Learning to Relocalize in Minutes Using RGB and Poses,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin B, 2023.
- [4] A. Kendall, M. Grimes, and R. Cipolla, “PoseNet: A Convolutional Network for Real-Time 6-DOF Camera Relocalization,” in *IEEE International Conference on Computer Vision (ICCV)*, mức tin A, 2015.
- [5] T. Sattler, Q. Zhou, M. Pollefeys, et al., “Understanding the Limitations of CNN-Based Absolute Camera Pose Regression,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, mức tin A; nguồn của khẳng định APR không tốt hơn image retrieval, 2019.
- [6] C. Campos, R. Elvira, J. J. Gómez Rodríguez, et al., “ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial, and Multimap SLAM,” *IEEE Transactions on Robotics*, vol. 37, no. 6, pp. 1874–1890, 2021, mức tin A.